Formative Assignment: Advanced Linear Algebra (PCA)

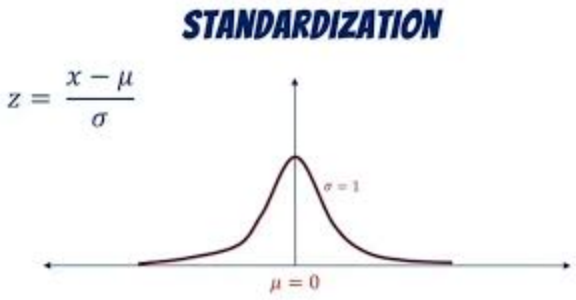
This notebook will guide you through the implementation of Principal Component Analysis (PCA). Fill in the missing code and provide the required answers in the appropriate sections. You will work with a dataset that is Africanized .

1. Make sure to display outputs for each code cell when submitting.
2. Do not write all your code on one cell
3. Do not use any libraries aside from numpy

Step 1: Load and Standardize the Data

Before applying PCA, we must standardize the dataset. Standardization ensures that all features have a mean of 0 and a standard deviation of 1, which is essential for PCA. Fill in the code to standardize the dataset.

STRICTLY - Write code that implements standardization based on the image below




```
import numpy as np
import csv

with open('Life-Expectancy-Data-Updated.csv', 'r') as f:
    reader = csv.reader(f)
    headers = next(reader)
    raw_data = [row for row in reader]

print("Columns:", headers)
print("Total rows:", len(raw_data))
```

```
Columns: ['Country', 'Region', 'Year', 'Infant_deaths', 'Under_five_deaths', 'Ad
Total rows: 2864
```

```
africa_rows = [row for row in raw_data if row[headers.index('Region')] == 'Afri
print(f"African rows: {len(africa_rows)}")
```

```
African rows: 816
```

```
non_numeric_cols = ['Country', 'Region']
numeric_cols = [h for h in headers if h not in non_numeric_cols]

numeric_indices = [headers.index(c) for c in numeric_cols]

numeric_data = []
for row in africa_rows:
    numeric_row = []
    for i in numeric_indices:
        val = row[i].strip()
        numeric_row.append(float(val) if val != '' else np.nan)
    numeric_data.append(numeric_row)

numeric_data = np.array(numeric_data, dtype=float)
print("Shape before handling NaN:", numeric_data.shape)
```

```
Shape before handling NaN: (816, 19)
```

```
col_means = np.nanmean(numeric_data, axis=0)
for col_idx in range(numeric_data.shape[1]):
    nan_mask = np.isnan(numeric_data[:, col_idx])
    numeric_data[nan_mask, col_idx] = col_means[col_idx]

print("NaN values remaining:", np.sum(np.isnan(numeric_data)))
```

```
NaN values remaining: 0
```

```
means = np.mean(numeric_data, axis=0)
stds = np.std(numeric_data, axis=0)
stds[stds == 0] = 1

standardized_data = (numeric_data - means) / stds
standardized_data[:5]
```

```
array([[-1.08465229e-01,  2.36695500e-01, -4.64399352e-02,
        -5.09468446e-01, -9.85465241e-01, -1.73344901e-01,
        -2.28385053e-01,  1.42739706e-01, -6.36063312e-02,
        -4.98873355e-02, -6.87266260e-01, -3.81598107e-01,
        -6.67279005e-01,  1.08533518e-02,  3.09271010e-04,
        -5.39692473e-01,  0.00000000e+00,  0.00000000e+00,
         3.50780550e-01],
       [ 9.76187060e-01, -8.21489142e-01, -8.06962168e-01,
        -5.01418301e-01,  1.56288695e+00,  2.27686316e-01,
        -2.28385053e-01,  9.39737949e-01,  2.00127237e-01,
         3.01910764e-01, -2.90096187e-01,  1.61778605e+00,
        -6.25664033e-01, -3.55110908e-01, -3.60212364e-01,
```

```
         1.58302129e+00,  0.00000000e+00,  0.00000000e+00,
         6.21302606e-01],
       [ 7.59256602e-01, -5.88459311e-01, -6.14646201e-01,
        -4.99753449e-01, -4.56299212e-01,  7.43297881e-01,
        -6.06401691e-01,  2.56596598e-01,  7.80341087e-01,
         7.54222606e-01, -4.54088991e-01, -2.43984634e-01,
         2.60622401e-01, -1.35532352e-01, -1.43899383e-01,
         1.08936693e+00,  0.00000000e+00,  0.00000000e+00,
         4.36855750e-01],
       [-9.76187060e-01,  1.52651955e-01,  1.34948988e-01,
         7.06482394e-01, -7.24363582e-01, -4.02505597e-01,
        -2.28385053e-01, -5.40401645e-01, -1.38227417e+00,
        -1.30630912e+00, -1.20978607e-01, -8.04439847e-02,
        -5.63429030e-01,  6.69589020e-01,  5.77143886e-01,
         5.46347127e-01,  0.00000000e+00,  0.00000000e+00,
        -4.97674991e-01],
       [ 7.59256602e-01, -5.65538344e-01, -5.90606705e-01,
        -6.56249560e-01, -7.27844937e-01, -2.87925249e-01,
        -2.28385053e-01, -1.22354300e+00, -1.69099758e-01,
        -1.50401078e-01, -6.53955222e-01, -6.14942692e-01,
         1.23780375e-01,  8.40462039e-02,  7.24135979e-02,
         7.43808873e-01,  0.00000000e+00,  0.00000000e+00,
         7.31970720e-01]])
```

Step 3: Calculate the Covariance Matrix

The covariance matrix helps us understand how the features are related to each other. It is a key component in PCA.

```
n = standardized_data.shape[0]
cov_matrix = (1 / (n - 1)) * (standardized_data.T @ standardized_data)
cov_matrix
```

```
 -1.64389830e-01,  7.69243911e-02, -7.50963819e-02,
  3.06823495e-02,  1.00122699e+00,  8.49442837e-01,
 -7.07883191e-02,  0.00000000e+00,  0.00000000e+00,
 -1.72537154e-01],
[-1.45654233e-01,  7.60511581e-02,  1.10680329e-01,
  1.63285938e-01,  1.42772735e-01, -1.34830615e-01,
 -1.00868014e-01, -1.35645721e-01, -1.19426199e-01,
 -1.34947669e-01,  1.32950872e-01, -3.18963308e-02,
  1.52010971e-02,  8.49442837e-01,  1.00122699e+00,
  7.63271254e-04,  0.00000000e+00,  0.00000000e+00,
 -1.65902572e-01],
[ 2.05590531e-01, -6.10424124e-01, -6.45714417e-01,
 -5.42444349e-02,  3.19449625e-01,  3.26689715e-01,
  2.41109382e-01,  6.15996869e-01,  3.66097998e-01,
  3.70890496e-01,  3.40142831e-01,  6.11334188e-01,
  1.01371305e-01, -7.07883191e-02,  7.63271254e-04,
  1.00122699e+00,  0.00000000e+00,  0.00000000e+00,
  3.68738949e-01],
[ 0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
  0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
  0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
  0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
  0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
  0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
  0.00000000e+00],
[ 0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
  0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
  0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
  0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
  0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
  0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
  0.00000000e+00],
[ 3.37348989e-01, -8.13301678e-01, -8.31757013e-01,
 -9.03019873e-01, -1.62490416e-01,  4.78599676e-01,
  4.23361682e-01,  5.03900442e-01,  5.42743795e-01,
  5.30072529e-01, -4.64615866e-01,  4.37626839e-01,
  7.46793022e-03, -1.72537154e-01, -1.65902572e-01,
  3.68738949e-01,  0.00000000e+00,  0.00000000e+00,
  1.00122699e+00]])
```

In a paragraph that has a maximum if 5 Lines, Explain why we need to compute a covariance matrix, provide atleast 2 reasons

The covariance matrix shows how the 19 health indicators are related to each other. It helps us identify indicators that move together, such as high infant mortality and low life expectancy. It is also important because PCA uses the covariance matrix to calculate the principal components

Step 4: Perform Eigendecomposition

Eigendecomposition of the covariance matrix will give us the eigenvalues and eigenvectors, which are essential for PCA. Fill in the code to compute the eigenvalues and eigenvectors of the covariance matrix.

```
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

eigenvalues = eigenvalues.real
eigenvectors = eigenvectors.real

eigenvalues, eigenvectors
```

```
  0.00000000e+00,  0.00000000e+00,  1.00000000e+00,
  0.00000000e+00],
[ 0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
  0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
  0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
  0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
  0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
  0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
  1.00000000e+00],
[ 3.37416156e-01, -2.48424982e-01,  2.25825932e-01,
  1.25571519e-02, -5.87552633e-02,  3.39337376e-02,
 -3.93741945e-02, -7.61254458e-02, -5.72556014e-02,
 -1.85324100e-01, -1.59544657e-03,  4.80727410e-02,
 -3.40932153e-03, -2.74906138e-01, -7.67649431e-01,
  2.27681037e-01,  6.84483070e-02,  0.00000000e+00,
  0.00000000e+00]]))
```

Step 5: Sort Principal Components

Sort the eigenvectors based on their corresponding eigenvalues in descending order. The higher the eigenvalue, the more important the eigenvector. Complete the code to sort the eigenvectors and print the sorted components.

How Is Explained Variance Used In PCA?'

```
sorted_indices = np.argsort(eigenvalues)[::-1]
sorted_eigenvalues = eigenvalues[sorted_indices]
sorted_eigenvectors = eigenvectors[:, sorted_indices]

explained_variance = sorted_eigenvalues / np.sum(sorted_eigenvalues)
cumulative_variance = np.cumsum(explained_variance)

for i, (ev, cum) in enumerate(zip(explained_variance, cumulative_variance)):
    print(f"PC{i+1}: {ev*100:.2f}% variance | Cumulative: {cum*100:.2f}%")

sorted_eigenvectors
```

```
PC1: 37.38% variance | Cumulative: 37.38%
PC2: 14.39% variance | Cumulative: 51.76%
PC3: 10.37% variance | Cumulative: 62.14%
PC4: 9.91% variance | Cumulative: 72.05%
PC5: 7.28% variance | Cumulative: 79.33%
PC6: 4.64% variance | Cumulative: 83.97%
PC7: 4.53% variance | Cumulative: 88.49%
PC8: 3.67% variance | Cumulative: 92.17%
PC9: 2.38% variance | Cumulative: 94.54%
PC10: 1.40% variance | Cumulative: 95.94%
PC11: 1.33% variance | Cumulative: 97.27%
PC12: 0.93% variance | Cumulative: 98.20%
PC13: 0.84% variance | Cumulative: 99.04%
PC14: 0.54% variance | Cumulative: 99.58%
PC15: 0.22% variance | Cumulative: 99.80%
```

```
PC16: 0.13% variance | Cumulative: 99.93%
PC17: 0.07% variance | Cumulative: 100.00%
PC18: 0.00% variance | Cumulative: 100.00%
PC19: 0.00% variance | Cumulative: 100.00%
array([[ 1.54362084e-01, -1.26280195e-01,  2.86304615e-02,
         4.84180439e-02,  4.49002655e-01, -7.29773055e-01,
         2.28543680e-01,  3.68842200e-01,  1.19293021e-01,
         1.12846313e-01,  2.65838366e-02,  6.37273872e-02,
         3.57441939e-02, -2.19689538e-02,  7.63255860e-03,
        -1.90155143e-02, -2.64508560e-02,  0.00000000e+00,
         0.00000000e+00],
       [-3.66825820e-01, -2.59703269e-02, -9.19865490e-02,
         6.36738839e-02, -6.16780806e-02, -4.06234156e-02,
        -1.02454778e-01,  7.44793405e-02,  3.55409069e-01,
         2.61667238e-01, -2.18271206e-01, -1.73911658e-01,
        -7.34444177e-02, -3.49508191e-01,  5.29462801e-02,
         6.52418203e-01,  4.55418127e-02,  0.00000000e+00,
         0.00000000e+00],
       [-3.74221342e-01, -1.88642282e-02, -8.86230899e-02,
         2.63751668e-02, -6.57546923e-02, -1.98618210e-02,
        -1.11513036e-01,  2.30401062e-02,  3.40186779e-01,
         2.23321183e-01, -1.18586093e-01, -2.35619104e-02,
         1.88626271e-03, -2.39102636e-01, -6.51899387e-02,
        -6.70994972e-01, -3.76156646e-01,  0.00000000e+00,
         0.00000000e+00],
       [-2.41293059e-01,  4.07562824e-01, -2.73980195e-01,
        -4.57563699e-03,  1.00417631e-01,  7.26316093e-03,
         1.58120768e-01,  9.34484862e-02, -1.79851493e-01,
         1.65940645e-01,  7.01949354e-02, -1.77460020e-01,
        -2.62953903e-02,  5.14486909e-01,  9.85577651e-02,
         1.73991914e-01, -5.05010474e-01,  0.00000000e+00,
         0.00000000e+00],
       [ 2.24526250e-04,  3.76430287e-01,  1.14023749e-01,
         1.96546409e-01,  1.87717406e-01, -2.43549019e-01,
        -7.58104314e-01, -5.20638645e-02,  6.98086444e-02,
        -3.31059124e-01, -8.18040356e-02,  5.57755882e-03,
        -6.81521862e-02,  8.97103281e-02,  1.91184349e-02,
         1.04276432e-02, -1.78388115e-02,  0.00000000e+00,
         0.00000000e+00],
       [ 2.94413078e-01,  5.52117892e-02, -2.65641462e-01,
        -2.99485430e-01,  3.26838915e-02,  6.35805135e-02,
        -1.77985662e-01, -1.16122992e-01,  2.16025455e-01,
```

Step 6: Project Data onto Principal Components

Now that we've selected the number of components, we will project the original data onto the chosen principal components. Fill in the code to perform the projection.

```
num_components = np.argmax(cumulative_variance >= 0.95) + 1
print(f"Number of components selected: {num_components}")

top_eigenvectors = sorted_eigenvectors[:, :num_components]
```

```
reduced_data = standardized_data @ top_eigenvectors
reduced_data[:5]
```

```
Number of components selected: 10
array([[-0.11277976, -1.25940502,  0.00384647, -0.49315544, -0.62598214,
        -0.07558793,  0.57702169, -0.30293898,  0.24427387, -0.21102631],
       [ 2.35136659,  0.91550852,  0.68694304,  1.4167418 ,  0.32703179,
        -1.36536502, -0.4709408 , -1.0024699 ,  0.06515391,  0.01371179],
       [ 1.73258378, -0.41214409, -0.1454901 , -0.4455028 ,  1.12390571,
        -0.02865567,  0.47125434, -0.43300909, -0.13422399,  0.33475807],
       [-1.74516957,  0.37935819,  0.79579632, -0.00336069, -0.91410249,
         0.24208181,  0.93393629, -0.59046117, -0.98022666,  0.2385298 ],
       [ 0.42766282, -1.2572314 ,  0.67198017, -0.39609927,  0.67673333,
        -0.41147591,  0.44633144, -0.02690071, -1.48419816,  0.0976907 ]])
```

Step 7: Output the Reduced Data

Finally, display the reduced data obtained by projecting the original dataset onto the selected principal components.

```
print(f'Original Data Shape: {standardized_data.shape}')
print(f'Reduced Data Shape: {reduced_data.shape}')
reduced_data[:5]
```

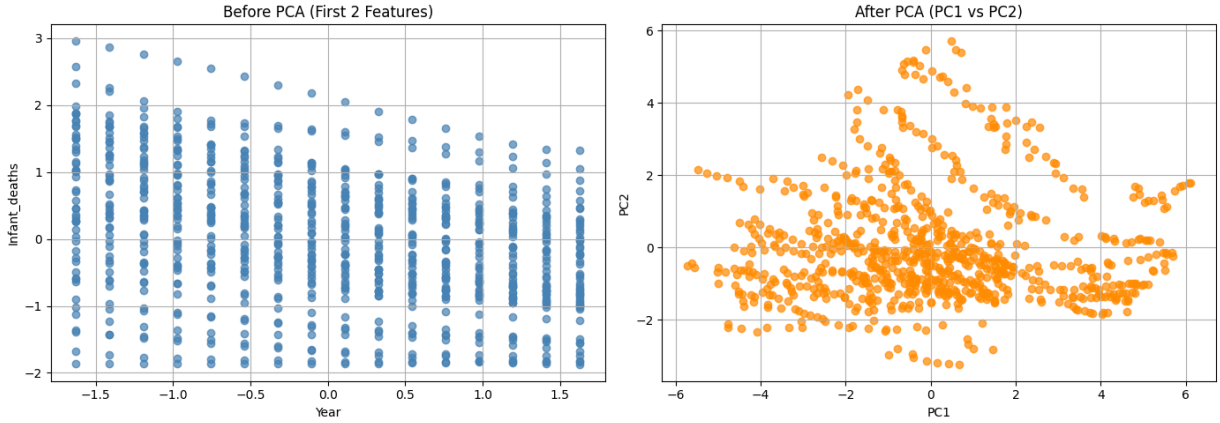
```
Original Data Shape: (816, 19)
Reduced Data Shape: (816, 10)
array([[-0.11277976, -1.25940502,  0.00384647, -0.49315544, -0.62598214,
        -0.07558793,  0.57702169, -0.30293898,  0.24427387, -0.21102631],
       [ 2.35136659,  0.91550852,  0.68694304,  1.4167418 ,  0.32703179,
        -1.36536502, -0.4709408 , -1.0024699 ,  0.06515391,  0.01371179],
       [ 1.73258378, -0.41214409, -0.1454901 , -0.4455028 ,  1.12390571,
        -0.02865567,  0.47125434, -0.43300909, -0.13422399,  0.33475807],
       [-1.74516957,  0.37935819,  0.79579632, -0.00336069, -0.91410249,
         0.24208181,  0.93393629, -0.59046117, -0.98022666,  0.2385298 ],
       [ 0.42766282, -1.2572314 ,  0.67198017, -0.39609927,  0.67673333,
        -0.41147591,  0.44633144, -0.02690071, -1.48419816,  0.0976907 ]])
```

Step 8: Visualize Before and After PCA

Now, let's plot the original data and the data after PCA to compare the reduction in dimensions visually.

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].scatter(standardized_data[:, 0], standardized_data[:, 1], alpha=0.7, cc
axes[0].set_title('Before PCA (First 2 Features)')
```

```
axes[0].set_xlabel(numeric_cols[0])
axes[0].set_ylabel(numeric_cols[1])
axes[0].grid(True)

axes[1].scatter(reduced_data[:, 0], reduced_data[:, 1], alpha=0.7, color='darkc
axes[1].set_title('After PCA (PC1 vs PC2)')
axes[1].set_xlabel('PC1')
axes[1].set_ylabel('PC2')
axes[1].grid(True)

plt.tight_layout()
plt.show()
```



Please make sure you do not use more than 5 lines to answer each of the following points:

1. Interpret the Visual you just created of the before and after PCA
2. Explain why you selected the number of principle components, more especially explain what tradeoffs you are making
3. Clearly mention based on your use case/ dataset ("economic activity", "population pressure"), What information is lost when reducing dimensions?

Before PCA, the data looks organised in vertical lines because the x-axis is just the year repeated many times. However, this does not show much useful information. After PCA, the data is spread across PC1 and PC2, which are combinations of all 19 health

indicators. The wider spread along PC1 shows that it captures most of the variation in the data, making the PCA plot more useful for understanding the overall patterns.

2.Why this number of components and tradeoffs

We reduced the data from 19 health indicators to 10 colums. We chose 10 because they keep 95% of the information in the dataset. The tradeoff is that 5% of the information is lost, including some small differences between countries. We accepted this because it makes the data much easier to analyze while still keeping the main health patterns across Africa.

3.What information is lost

When we reduce the dimensions using PCA, we lose some of the smaller details in the data. For example, a country may have high school attendance but poor HIV outcomes. These unique country differences get combined into the principal components and are harder to see on their own. PCA helps us see the overall pattern in the data, but some specific country-level details are lost.

Double-click (or enter) to edit